



BSc (Hons) in **Information Technology**  
**Specialized in AI**

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 02

**Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

### Part 1: Practical Implementation — Coding the Architectures

#### Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

#### Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*  
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

### Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.  
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

## Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

**1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?**

For a complex environment such as chess, a perfect table-driven agent is not actually practical because for every possible sequence of perception, it would have to record an action. The number of possible perception sequences increases extremely fast as the agent keeps operating for a longer time. This results in a

**2. (Understand) Look at the code you wrote for your SimpleReflexAgent . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.**

Inside the sense\_and\_act() function, we can see statements such as if, elif, and else, which show the Condition-Action-Rule. Each condition selects a suitable action based on what the agent currently observes. The code directly follows the condition-Action structure of a simple reflex agent here. For example, when food

**3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?**

The Simple Reflex Agent can enter an infinite loop because it makes decisions only from the current perceptions and can not remember what happened before. Because the environment is observable partially, the same perception might be received by the agent in different locations. As the agent has no memory of

**4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).**

The Model-Based Agent, unlike the Simple Reflex Agent, uses memory to store information about the environment and previous actions. The sensor Model uses the current perception, while the Transition Model updates the internal state based on the previous action. This helps the agent choose better actions by using both

Finalize and Lock Lab 02 Documentation

**Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.**



## **FACULTY OF COMPUTING**